

Presenter's Guide

Hardware/Software Co-Design for LLM Quantization
Gyanu Pandey • CECS 528 • December 2024

⚠ **SAY THIS AFTER SLIDE 1**

"So before I dive in, let me clarify what this is. This is a **comparative analysis** — I'm not building anything here. I'm looking at two hardware approaches for LLM quantization, Oltron and FlightLLM, and comparing them to see how they stack up against software-only methods."

SLIDE 1 — Title

"Alright, so today I'm talking about Hardware-Software Co-Design for LLM Quantization. Basically — how do we make these massive language models actually run efficiently?"

→ "Let me give you the context..."

Hardware/Software Co-Design for LLM Quantization

Combining H/W and S/W Approaches for Efficient Inference

Speaker Profile

Dr. [Name], Assistant Professor, [Institution]

[Email]

[Website]

SLIDE 2 — Introduction & Background

"So here's the thing. LLMs are everywhere now — GPT-4, LLaMA, Claude — they're doing incredible stuff with text, code, reasoning. But they're massive. Like, really massive."

"And that's the problem. They don't fit. So we need to shrink them — that's quantization. But if we just shrink them naively, they break. So the question is: can we shrink them AND make them run fast? That's where hardware co-design comes in."

→ "But why does any of this matter? Let me show you the numbers..."



SLIDE 3 — Motivation

🔥 START WITH THIS

"So I want to show you three numbers that kind of blew my mind when I first saw them..."

"\$700K per day. That's what it costs to run ChatGPT-scale inference. Per day. That's not a typo."

"300 watts per GPU — that's an A100 running 24/7. Now imagine thousands of them in a datacenter."

"And 140 gigs — that's just to load LLaMA-70B into memory. My laptop has 16. It doesn't fit."

"So this isn't just an academic problem. If you can't deploy efficiently, you can't deploy at all."

→ "So let me show you exactly what this memory problem looks like..."



SLIDE 4 — The Problem: Memory Wall

🚚 REAL-LIFE EXAMPLE — START WITH THIS

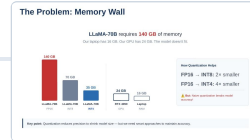
"Imagine you have a truck that's 140 feet long, and your garage is 24 feet. Doesn't matter how good a driver you are — it just won't fit. That's exactly what's happening here."

"So LLaMA-70B needs 140 gigs just to load the weights. My laptop has 16. A 4090 — which is like a \$1600 GPU — has 24. Even an A100, the big datacenter GPU, only has 80. The model just doesn't fit."

"What can we do? We shrink it. INT8 cuts it in half — 70 gigs. INT4 gets us to 35. That's actually starting to fit on some hardware."

"But here's the catch — if we just blindly cut precision, the model starts outputting garbage. So we need to be smart about it."

→ "So what exactly is quantization? Let me break that down..."



SLIDE 5 — What is Quantization?

🏠 REAL-LIFE EXAMPLE

"Think about JPEG compression. You take a raw photo — huge file, every pixel perfect — and you compress it. You lose some detail, but for Instagram? It's fine. Quantization is basically the same idea but for neural network weights."

"So quantization just means reducing the precision. Instead of 32-bit floats, we use 8-bit or even 4-bit integers."

"FP32 is the baseline — full precision. INT8 is 4x smaller. INT4 is 8x smaller."

"The benefits are obvious — smaller memory, faster inference, lower power. But the challenge is: if you do this naively, you destroy the accuracy. The model just breaks."

→ "But here's the real question — why isn't software quantization enough?"



SLIDE 6 — Why Hardware Matters

🔗 REAL-LIFE EXAMPLE

"It's like having a Ferrari engine but driving on a dirt road. The A100 has 312 TFLOPS of compute — that's insane. But it's just sitting there waiting for data from memory. The road is the bottleneck, not the engine."

"Here's the thing most people don't realize — LLM inference is memory-bound, not compute-bound. To generate just one token, you have to read all 70 billion weights. Even at INT4, that's 35 GB per token."

"So software quantization saves memory, sure. But GPUs still have to dequantize everything back to FP16 to actually compute. You're still bottlenecked on memory bandwidth."

"What we really need is hardware that can do native INT4 and INT8 operations — no dequantization, no overhead."

→ "Let me show you how software approaches currently work..."



SLIDE 7 — Software-Only Approach

"So here's how software quantization works on a standard GPU. You store the weights as INT8, then at runtime you dequantize them back to FP16, do the computation, get your output."

"The popular tools — bitsandbytes, ONNX Runtime, llama.cpp, AutoGPTQ — they all do this. They run on standard NVIDIA GPUs."

"But see that dequantization step? That's the problem. Every single operation has to convert INT8 back to FP16. And that overhead? It adds up fast."

→ "How bad is the overhead? Let me show you some benchmarks..."



SLIDE 8 — Software Limitations

🔗 REAL-LIFE EXAMPLE

"It's like vacuum-packing your clothes for a trip. You save space in the suitcase, great. But at the hotel, you have to unpack and re-inflate everything before you can wear it. If you're changing outfits constantly, the unpacking overhead kills any space savings you got."

"So CipherCore ran benchmarks, and the results are kind of surprising. bitsandbytes? Actually 35% slower than baseline. Not faster — slower."

"ONNX Runtime does better — 1.7× faster. That's the best software result I found."

"llama.cpp on CPU? 30% slower."

"So why is software quantization sometimes making things worse? Dequantization overhead on every op, memory bandwidth is still the bottleneck, and you can't skip the zeros."

→ "This is why I started looking at hardware solutions..."



The screenshot shows a table titled "Software-Only Limitations" with a subtitle "Comparing performance of different quantization methods". The table has columns for "Method", "Speedup", "Memory", and "Latency". The rows are: "Baseline", "bitsandbytes", "ONNX Runtime", and "llama.cpp".

Method	Speedup	Memory	Latency
Baseline	1.0x	~100MB	~100ms
bitsandbytes	~0.65x	~100MB	~100ms
ONNX Runtime	~1.7x	~100MB	~100ms
llama.cpp	~0.7x	~100MB	~100ms

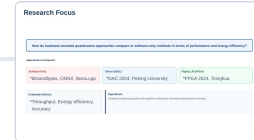
SLIDE 9 — Research Focus

"So here's the question I'm trying to answer: Can hardware-assisted quantization actually beat software-only methods? And by how much?"

"I'm comparing three things. Software — bitsandbytes, ONNX, llama.cpp. Oltron — that's a custom ASIC from Peking University, published at DAC 2024. And FlightLLM — that uses FPGAs, from Tsinghua, published at FPGA 2024."

"My hypothesis? Hardware will significantly outperform software because it eliminates that dequantization overhead completely."

→ "Alright, let's start with Oltron..."



SLIDE 10 — Oltron: The Outlier Problem

REAL-LIFE EXAMPLE — GRADING ANALOGY

"Think about grading on a curve. 99 students score between 60 and 80. But one student scores 200 — total outlier. Do you scale for the majority, and that outlier breaks everything? Or scale for the outlier, and suddenly everyone else looks the same? You can't win with a single scale."

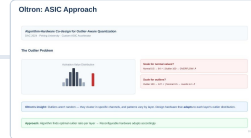
"So Oltron is from Peking University, published at DAC 2024. It's a custom chip — an ASIC."

"The problem they're solving: in neural networks, 99% of values are small, clustered around zero. But about 1% are these huge outliers — 10 to 100 times bigger."

"If you scale for normal values, outliers overflow and break. Scale for outliers, you lose precision on everything else. Classic dilemma."

"What Oltron figured out is that outliers aren't random. They cluster in specific channels, and the pattern is different for each layer. So their solution — don't use one fixed ratio. Build hardware that adapts to whatever each layer needs."

→ "Let me show you their three innovations..."



SLIDE 11 — Oltron: Key Innovations

"So here's how their chip works. Three innovations."

"First — adaptive quantization. They analyze each layer, figure out where the outliers are, and allocate precision accordingly. Some channels get INT4, others get FP16. Layer 1 might need 2% FP16. Layer 5 might need 15%. It varies."

"Second — reconfigurable processing elements. Unlike a GPU where the cores are fixed, Oltron can actually switch between INT4 and FP16 operations dynamically. The hardware reconfigures for each layer."

"Third — tile-based dataflow. They reorganized how data moves to maximize reuse and minimize bandwidth."

"Results? 1.9× faster than the previous best. 1.6× more energy efficient. And less than 1% accuracy loss. That's the power of co-design."

→ "Now let me show you the detailed results..."



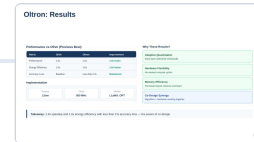
SLIDE 12 — Oltron: Results

"So to summarize Oltron — 1.9× speedup. 1.6× more energy efficient. Less than 1% accuracy loss."

"They built this as a custom ASIC in 12nm, running at 500MHz. Tested it on LLaMA and OPT."

"The takeaway here is that co-design works. Algorithm and hardware together can do things neither can do alone."

→ "Now let's look at a completely different approach — FlightLLM uses FPGAs instead..."



Oltron: Results

Performance of the Oltron ASIC				ASIC Test Results	
Model	Throughput (tokens/sec)	Energy (mJ/token)	Accuracy (%)	Throughput (tokens/sec)	Energy (mJ/token)
LLaMA	100	1.0	99.9	190	0.6
OPT	100	1.0	99.9	160	0.6

1.9x speedup, 1.6x more energy efficient, less than 1% accuracy loss.

SLIDE 13 — FlightLLM: FPGA Approach

🏭 REAL-LIFE EXAMPLE — FACTORY VS WORKSHOP

"A GPU is like a factory with fixed assembly lines. Super efficient for standard products, but you can't skip steps even when they're useless. An FPGA is like a custom workshop — you design your own machines for exactly what you need. More flexible, can skip useless steps."

"FlightLLM is from Tsinghua, published at FPGA 2024. So why FPGAs instead of GPUs?"

"Three problems with GPUs. First — fixed architecture. They can't skip zeros. Even if half your weights are zero, the GPU still processes them. Second — memory bottleneck. You're constantly pulling data from slow DRAM. Third — power. An A100 uses 300 watts."

"FPGAs are different. They're reconfigurable — you can build custom circuits that skip zeros entirely. They have fast on-chip SRAM — 100× faster than DRAM. And the whole thing runs on just 75 watts."

→ "Here are their three innovations..."



SLIDE 14 — FlightLLM: Key Innovations

"Three innovations here too."

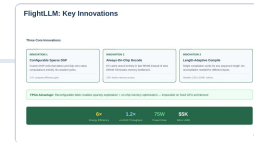
"First — configurable sparse DSP. They built custom DSP units that detect zeros and skip them entirely. No wasted cycles. That's 1.6× compute efficiency right there."

"Second — always-on-chip decode. The KV cache stays entirely in SRAM instead of DRAM. That's 100× faster memory access. No more waiting for data."

"Third — length-adaptive compilation. You compile once, and it works for any sequence length. You don't need to recompile for different inputs."

"Results? 6× energy efficiency. 1.2× better throughput than an A100. At 75 watts on a \$5K FPGA."

→ "Let me show you the numbers..."



SLIDE 15 — FlightLLM: Results

KEY INSIGHT

"Same throughput as a \$15,000 A100 GPU, at 1/4 the power, on a \$5,000 FPGA. At datacenter scale, that power difference is everything."

"So here's the comparison. V100S — 250 watts, baseline performance. A100 — 300 watts, slightly faster. FlightLLM — 75 watts, and it actually beats the A100 in throughput."

"That's 6× better energy efficiency. Same throughput as a \$15,000 GPU, at one-fourth the power, on a \$5,000 FPGA."

"Why does this matter? At datacenter scale, power is everything. If you can do the same work with 1/4 the electricity, that's huge savings."

→ "Now let me put it all together..."



SLIDE 16 — Comparison: Software vs Hardware

"So let's compare everything side by side."

"Software-only? 0.65× to 1.7× speedup. Sometimes it's actually slower than baseline."

"Oltron ASIC? 1.9× speedup, 1.6× energy efficiency."

"FlightLLM FPGA? 1.2× vs A100 throughput, 6× energy efficiency."

"The pattern is clear. Software is limited by fixed hardware — dequantization overhead, can't exploit sparsity. Hardware approaches can do native INT4/INT8, they're reconfigurable, and memory is co-optimized."

"Trade-off? Software is more accessible — just run pip install. Hardware takes real investment. But for production at scale? Hardware wins."

→ "Before I conclude, let me talk about future work..."

Comparison: Software vs Hardware

Metric	Software (A100)	Hardware (ASIC/FPGA)
Speedup	1.0x	0.65x - 1.7x
Energy Efficiency	1.0x	1.6x - 6x
Throughput	1.0x	1.2x - 1.9x
Reconfigurability	No	Yes
Memory Co-optimization	No	Yes

Table 1: Comparison of Software vs Hardware Performance. The table shows that hardware approaches (ASIC/FPGA) generally outperform software (A100) in terms of speedup, energy efficiency, and throughput. Hardware also offers reconfigurability and memory co-optimization, which software lacks.

SLIDE 17 — Future Work

"So three directions I'm thinking about."

"First — hands-on benchmarking. I want to validate those CipherCore numbers myself. Run bitsandbytes and llama.cpp, see what I get."

"Second — emerging hardware. Groq's LPU, AWS Inferentia, Google TPU — they all support quantized inference natively. That's worth exploring."

"Third — cost-benefit at scale. GPU rental vs FPGA vs ASIC — what's the total cost of ownership?"

"These all connect back to what we've been learning in class — memory hierarchy, dataflow architecture, hardware-software co-design."

→ "Let me wrap up..."



SLIDE 18 — Conclusion

"Three takeaways."

"One — hardware matters. LLM inference is memory-bound. Custom hardware eliminates that dequantization overhead."

"Two — co-design wins. Algorithm and hardware together can do things neither can do alone. Oltron's adaptive quantization with reconfigurable PEs. FlightLLM's sparse DSPs with on-chip memory."

"Three — energy is key. FlightLLM's 6× efficiency matters at datacenter scale. That's the difference between profitable and not."

"Remember these numbers: 140 GB for LLaMA-70B. 1.9× Oltron speedup. 6× FlightLLM efficiency. 75 watts vs 300 watts."

→ "I'm happy to take questions..."



SLIDE 19 — Any Questions?

"Thanks for listening. Happy to take any questions — about the hardware approaches, the benchmarks, or how this connects to what we've been learning."

[If no questions:] "Anyone curious why FPGAs might be better than ASICs for research? Or about the specific innovations in either paper?"



Quick Reference: Key Numbers

Metric	Value
LLaMA-70B memory	140 GB
Oltron speedup	1.9×
Oltron energy efficiency	1.6×
FlightLLM energy efficiency	6×
FlightLLM vs A100 throughput	1.2×
FPGA power	~75W
A100 power	~300W
Accuracy loss	<1%

Anticipated Q&A

Q: Why not just use bigger GPUs?

A: Even H100 with 80GB can't fit the biggest models. And for models that do fit, you're still memory-bound. Custom hardware solves the fundamental bottleneck.

Q: ASICs or FPGAs — which is better?

A: ASICs are more efficient but expensive and inflexible. FPGAs let you update as models evolve. For research, FPGAs make sense. For hyperscale production, ASICs pay off.

Q: What about accuracy loss?

A: Both papers report <1% degradation vs FP16. The key is adaptive quantization — treating layers differently based on sensitivity.

Q: Why is software quantization sometimes slower?

A: Dequantization overhead. Every op converts INT8→FP16→compute→back. This can outweigh memory savings, especially for small batches.

Q: Cost difference between FPGA and GPU?

A: Xilinx U280 ~\$5K vs A100 ~\$15K+. But power costs matter more at scale — FlightLLM uses 1/4 the power for similar throughput.